

# TTP Summer 2026 Bootcamp — Course Outline

## TTP Summer 2026

### 12-Week Bootcamp — Course Outline

#### *Rough Draft*

#### Overview

Students entering this cohort are expected to have a foundation in computer science through at least their sophomore year. The curriculum builds on that baseline, focusing on real-world tools, systems, and practices students are unlikely to encounter in an academic setting. Full-stack development remains a core thread, but the program also integrates AI-assisted coding throughout, introduces data science and visualization, covers networking and systems fundamentals, and takes a practical approach to cybersecurity — emphasizing secure development and SOC-relevant skills over theoretical concepts. Secure coding is woven into the curriculum from Week 2 onward rather than treated as a standalone topic. The final three weeks are reserved for a student-driven capstone project.

#### Curriculum Areas

- Full-Stack Web Development (Weeks 2–5)
- AI-Assisted Coding — integrated throughout
- Secure Development — integrated from Week 2
- Data Science: Collection, Cleaning & Visualization (Weeks 6–7)
- Networking, Systems & Security (Weeks 8–9)
- Student-Driven Capstone Project (Weeks 10–12)

#### Week 1: Environment Setup & Developer Foundations (1-5 June)

- Dev environment setup: VS Code, extensions, terminal, Git & GitHub
- OS-level configuration: shell profiles, PATH, SSH keys
- Introduction to AI-assisted coding: GitHub Copilot, Claude, prompt basics
- Using AI tools effectively and critically — when to trust, when to verify
- Command line fundamentals: navigation, file management, scripting basics

- Version control workflow: branching, commits, pull requests
- Introduction to the bootcamp tech stack and learning roadmap

#### Week 2: Web & JavaScript in Professional Practice (8-12 June)

- HTML/CSS in a real codebase: semantic structure, Flexbox, Grid, responsive patterns
- JavaScript ES6+ applied: modules, destructuring, spread, async/await in context
- DOM manipulation and event handling beyond textbook examples
- Browser DevTools as a real debugging workflow, not just an inspector
- Code readability, naming conventions, and style standards used in industry
- Secure coding intro: never trust user input, output encoding, avoiding common JS pitfalls
- AI pair programming: using Copilot and Claude effectively without over-relying on them
- Project: Build or substantially refactor a small interactive web app

#### Week 3: APIs, Data & Backend Foundations (15-19 June)

- REST principles, HTTP methods, and status codes in practice
- Fetching and handling data with Axios and the Fetch API
- JSON structure, parsing, and real-world data handling patterns
- API authentication: API keys, OAuth basics, environment variables
- Intro to Node.js and Express.js: building a simple server
- Secure API design: input validation, rate limiting, error handling that doesn't leak info
- Using AI to scaffold API integrations and write boilerplate
- Project: Weather app or public API integration

#### Week 4: Full-Stack Development: Databases & Server-Side Logic (22-26 June)

- SQL in practice: CREATE, INSERT, SELECT, UPDATE, DELETE
- PostgreSQL setup, pgAdmin, relationships and joins
- ORM concepts with Sequelize or Prisma
- Building RESTful APIs with Express + PostgreSQL
- Secure database practices: parameterized queries, least privilege, avoiding SQL injection

- Environment variables, secrets management, and keeping credentials out of code
- AI-assisted database query generation and debugging
- Project: Full-stack CRUD application

#### Week 5: React.js & Modern Frontend 29 June – 3 July)

- React fundamentals: JSX, components, props, state
- React Hooks: useState, useEffect, custom hooks
- Component lifecycle and data flow
- React Router for multi-page SPAs
- Styling in React: Tailwind CSS or CSS Modules
- Frontend security: XSS prevention, safe rendering, dependency awareness
- AI-assisted React: generating components, refactoring with LLMs
- Project: Full-stack React app connected to a backend API
- Capstone: Form groups and begin informal ideation — proposals due Week 8

#### Week 6: Data Science Foundations: Collection, Cleaning & Analysis (6-10 July)

- Intro to data science workflow: collect, clean, analyze, visualize
- Python for data: Pandas and NumPy fundamentals
- Data collection: web scraping, CSV/JSON ingestion
- Data cleaning: handling nulls, duplicates, type casting, normalization
- Exploratory data analysis (EDA) techniques
- Data privacy considerations: PII, responsible handling, basic compliance awareness
- Using AI to assist with data wrangling scripts and pattern recognition
- Project: Clean and analyze a real-world dataset

#### Week 7: Data Visualization & Interpretation (13-17 July)

- Data visualization principles
- Matplotlib and Seaborn for Python-based charts
- Interactive visualizations with Plotly or D3.js basics
- Building dashboards: Streamlit or basic React + Recharts
- Communicating findings: storytelling with data
- AI-assisted chart generation and interpretation

- Project: Data dashboard presenting insights from Week 6 dataset

#### Week 8: Networking, Systems & Security Fundamentals (20-24 July)

- Networking in practice: OSI model, TCP/IP, DNS, HTTP/HTTPS
- How browsers communicate: requests, headers, cookies, sessions
- Linux/Unix systems: permissions, processes, file system
- Authentication & authorization deep dive: JWT, sessions, OAuth flows, hashing and salting
- OWASP Top 10: understanding common vulnerabilities and how secure developers prevent them
- Logging, monitoring basics, and why they matter for both ops and security
- Hands-on: intentionally vulnerable app exercise (e.g. DVWA or similar)
- Capstone: Project proposals due — groups present ideas for instructor approval

#### Week 9: Cybersecurity Roles & the Security Mindset (28-31 July)

- How security teams are structured: SOC analysts, threat intelligence, incident response, AppSec
- What a SOC analyst actually does day-to-day and what skills transfer from development
- Threat intelligence fundamentals: indicators of compromise, threat feeds, basic OSINT
- Incident response lifecycle: detect, contain, eradicate, recover
- Security as a developer: DevSecOps principles and shifting security left
- AI in security operations: log analysis, anomaly detection, alert triage
- Discussion: realistic entry points into security roles from a dev background
- Mini CTF/Hackathon for demonstrated knowledge

#### Week 10: Capstone Project — Kickoff & Sprint 1 (3-7 Aug)

- Approved projects begin: finalize scope, architecture, and team roles
- Requirements gathering, user stories, and project roadmap
- Tech stack justification and instructor sign-off
- Git workflow for teams: branching strategy, code reviews, PRs
- Daily standups and agile sprint planning
- Instructor check-ins: architecture review and course correction

#### Week 11: Capstone Project — Sprint 2 (10-14 Aug)

- Continued development: core features and system integration
- Mid-point review: instructor feedback, scope adjustment if needed
- Testing and QA: bug triage, edge cases, peer code review
- Documentation: README, API docs, and inline comments
- Daily standups and sprint retrospective

#### Week 12: Capstone Project — Sprint 3 & Demo Day (17-21 Aug)

- Final sprint: feature completion, polish, and hardening
- Deployment: hosting on Railway, Vercel, Render, or similar
- Presentation preparation: technical walkthrough and demo script
- Demo Day: student presentations to instructors and guests
- Retrospective: what was built, what was learned, what comes next